Claude Code & OpenClaw

by Jack Yansong Li

Liii Network & Eastern Institute of Technology

Email: yansong@liii.pro

Table of contents

1 What is a LLM Agent? 2
- 1.1 Workflow (Optional) 2
  - Workflow products: 2
- 1.2 Coding Agent 2
  - Coding agent products: 2

2 Shell Basics 2

3 ReAct (Optional) 3
- 3.1 ReAct v.s. workflow 3
  - What is ReAct? 3
  - ReAct vs. Workflow: The Key Difference 4

4 Demo: local agent server 4
- 4.1 Project Overview 4
- 4.2 Technology Stack 4

5 Architecture and Design Patterns 5
- 5.1 High-Level Architecture 6
- 5.2 Streaming Architecture 6
  - 5.2.1 Multi-Type Streaming System 6
  - 5.2.2 Streaming Implementation 6
  - 5.2.3 Registry Pattern: Tool Management 6

Appendix A Homework 7

Appendix B Shell command basics 7

Appendix C Version Control 7

1 What is a LLM Agent?

We give an informal definition for LLM agent:

Informal Definition 1. A LLM agent is defined as:

$$\text{Agent}=\text{Context}+\text{LLMs}+\text{Tool Use}.$$

In practice, we can also classify LLM agents into therapy agent, service agent, coding agent... We will focus mostly on coding agent in this short lecture. However, we take a Customer Service Agent as our first example by its simplicity.

Example 1. (Customer Service Agent)

- Context: Product Manual. New Employee Training Manual...
- LLMs: Any LLMs..

- Tool Use: call transfer, web link…

Informal Definition 2. (Long-term Memory)

Long-term Memory=Context Window=Input Token Limit

1.5 Token≈1 Word

- **Question 1.** What's gonna happen if we put an entire Product Manual as a context? Will the LLM behave properly?

Example 2. (Claude Code)

- Context: Code, Documents, *System Prompts*
- LLMs: By default Claude Sonnet & Opus. By using *Claude Code Router*, one can apply any LLM through keys.
- Tool: Shell command

Example 3. (Cline, Copilot, Trae, Cursor)

- Context: Code, Documents, *System Prompts*
- LLMs: Any LLMs
- Tool: Shell command (through VSCode, unstable, failed sometimes!), VSCode commands

- **Question 2.** Claude Code and Cursor can use the same model, why the price is different?

1.1 Workflow (Optional)

> in one sentence, introduce me the difference between LLM coding agent and LLM workflow

An LLM coding agent is an autonomous system that executes programming tasks end-to-end, while an LLM workflow is a structured sequence of steps, often involving human oversight, to guide an LLM toward a specific outcome.

Workflow products:

- Dify (1 year free for university student, web-based).
- LangChain (require manual coding).

1.2 Coding Agent

Coding agent products:

- Cursor/Trae/Cline/Copilot: in VSCode
- Claude Code: in Shell
- OpenClaw: in Shell

2 Liii STEM basics

2.1 Short tutorial on Liii STEM

- LEGO symbol and Tab Cycling.
- Interacting with AI using DeepSeek.

2.2 Download, Install, register, and use!

Hands-Dirty Exercise 1. Join QQ group with group number 1080338876, download Liii STEM from group files.

Hands-Dirty Exercise 2. Register Liii STEM with `dku.edu` email.

Hands-Dirty Exercise 3. Ask DeepSeek in Liii STEM to solve

$$i\hbar\frac{\partial\psi}{\partial t} = \hat{\mathcal{H}}\psi \tag{1}$$

for Hydrogen atom. Please type (1) by yourself! Remember to store it with name `schrodinger.tmu`.

3 Shell Basics

Claude Code and OpenClaw lives in Shell (recall Figure 1), so the tool is your shell commands! Learn some terminal commands!

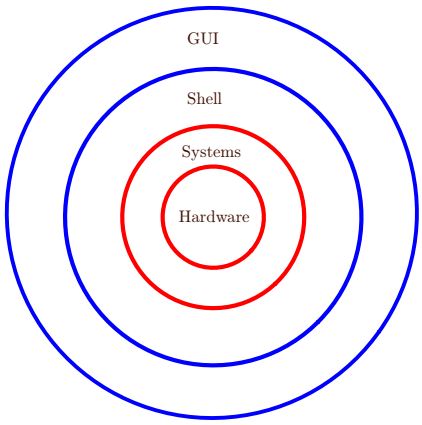

Figure 1. Different level of computer interactions

Remark 1. For a more tutorials on shell commands and command line tools, I highly recommend the *Missing Semester* Course provided by MIT: https://missing.csail.mit.edu/

Figure 2. Terminal (Physical)

- **Question 3.** What is the difference between shell, terminal, terminal emulator, TUI, and GUI?

3.1 Shell command

See Missing Semester Lecture 1. Most common shell "languages" are:

- Windows: Command Prompt, Powershell, Elvish.
- MacOS: Zsh, Elvish.
- Linux: Zsh, Bash, Fish, Elvish.

(Bash) Shell command includes make folder: `mkdir`, print working directory: `pwd`, list contents in the current working directory: `ls`, etc

```
Elvish] ls
Elvish] pwd
  /Users/jackyansongli
Elvish]
```

Hands-Dirty Exercise 4. In Powershell or Terminal, make a directory named `tuto_liii` on your home directory and create a file named `README.md`, put a `Hello world` in the first line of this file. Move `schrodinger.tmu` to the root of `tuto_liii`.s

3.2 Install software on your Shell

Hands-Dirty Exercise 5.

1. Install Scoop (Windows) or Homebrew (macOS): `https://scoop.sh/` or `https://brew.sh/`
2. Install Claude Code with Scoop

   Windows

   ```
   scoop install git
   setx CLAUDE_CODE_GIT_BASH_PATH "%USERPROFILE%\scoop\apps\git\current\usr\bin\bash.exe"
   irm https://claude.ai/install.ps1 | iex
   ```

   macOS

   ```
   brew install --cask claude-code
   ```

3. Configure Claude Code with Kimi K2.5: Kimi K2.5 on Claude Code. (You are also welcome to use other APIs)

3.3 Claude code basics

- Slash command `/init`, `/status`, `/compact`...

Hands-Dirty Exercise 6. Ask Claude Code to translate `schrodinger.tmu` into `schrodinger.md`.

4 Memory is all you need, use OpenClaw

4.1 OpenClaw workspace

OpenClaw stores its memory on `~/.openclaw/workspace`.

- `Heartbeat.md`

- `SOUL.md`
- `IDENTITY.md`
- ⋮

4.2 Install OpenClaw!

Hands-Dirty Exercise 7. Install OpenClaw on Windows or macOS

- for macOS: `curl -fsSL https://openclaw.ai/install.sh | bash`

During the installation, please choose moonshot and use `kimi k2.5` API with `mooonshot.cn` platform. No skills are required. If you have a Feishu account, you can also install the feishu skill.

Some Windows users may encounter issue on installation, there is a serious bug on Windows for the new version of OpenClaw, please follow

1. `npm -g uninstall openclaw`
2. `npm -g install openclaw@2026.4.2`
3. `openclaw onboard --install-daemon`
4. `openclaw gateway --port 18789 --verbose`

to install an earlier version of OpenClaw.

Hands-Dirty Exercise 8. Configure OpenClaw so you can use it on Liii STEM. Follow the instruction on https://h02wf0jq8yp.feishu.cn/wiki/BpWFwLUttin0w7keHcpcsBiOnIe, The instruction is in Chinese, you can use translation tools.

Hands-Dirty Exercise 9. Ask OpenClaw to read `schrodinger.tmu` and memorize the format of `.tmu`. Also ask him where he stored his memory.

Hands-Dirty Exercise 10. Now, OpenClaw should be able to create `.tmu` files by his own. Choose whatever topics you want and work with him to create a short report. If you have no idea, you can also choose from the following topics:

- A short tutorial on Quantum mechanics.
- A short tutorial on Linear regression.
- A short tutorial on Machine Learning.

Appendix A ReAct Architecture (Optional)

A.1 Workflow (Optional)

An LLM coding agent is an autonomous system that executes programming tasks end-to-end, while an LLM workflow is a structured sequence of steps, often involving human oversight, to guide an LLM toward a specific outcome.

Workflow products:

- Dify (1 year free for university student, web-based).
- LangChain (require manual coding).

A.2 ReAct v.s. workflow

What is ReAct?

ReAct (Reason + Act) is a method for AI agents to solve problems by mimicking human thought:

1. **Reason:** Think step-by-step about what to do next.
2. **Act:** Use a tool (like Search or an API) to do it.
3. **Observe:** See the result.

This loop repeats until the problem is solved. It's dynamic and flexible.

Example:

- **Thought:** "I need the capital of France."
- **Act:** `Search("capital of France")`
- **Observe:** "Paris."
- **Answer:** "The capital is Paris."

ReAct vs. Workflow: The Key Difference

	ReAct	Workflow (e.g., Zapier)
Nature	**Dynamic.** Figures out the path as it goes.	**Static.** Follows a pre-defined path.
Decision	Makes decisions **in real-time** based on reasoning.	All decisions are **pre-programmed** (if/else).
Best For	Open-ended, complex tasks (research, analysis).	Repetitive, predictable tasks (send email on form submit).
Analogy	A **Human Assistant** you give a goal to.	A **Factory Robot** on an assembly line.
Products	Claude Code	Dify

In short: ReAct *figures out* the steps, a Workflow *follows* the steps.

> introduceme ReAct, and what is the difference between react and workflow

>

Appendix B Demo: local agent server

Demo Github: https://github.com/JackYansongLi/local_llm_server.

B.1 Project Overview

The Demo is an educational project that demonstrates advanced LLM tool calling functionality across multiple platforms. This project serves as a comprehensive example of modern AI agent development, showcasing:

- **Tool Calling**: Standard function calling format with extensible tool registry
- **Real-time Streaming**: Multi-phase streaming with visual feedback for different content types
- **Cross-Platform Compatibility**: Universal deployment across Windows, macOS, and Linux
- **Modern Development Practices**: Comprehensive testing, configuration management, and deployment strategies

B.2 Technology Stack

See Table 1.

Component	Technology	Purpose
LLM platform	Ollama (>=0.1.0)	CPU-based local inference
Package Management	UV	Fast, reliable dependency management
Model	Qwen3-0.6B	Tool-optimized language model
Testing	pytest	Comprehensive test suite

Table 1. Technology Stack Overview

B.3 High-Level Architecture

The system employs a **Universal Agent Pattern** with intelligent backend selection:

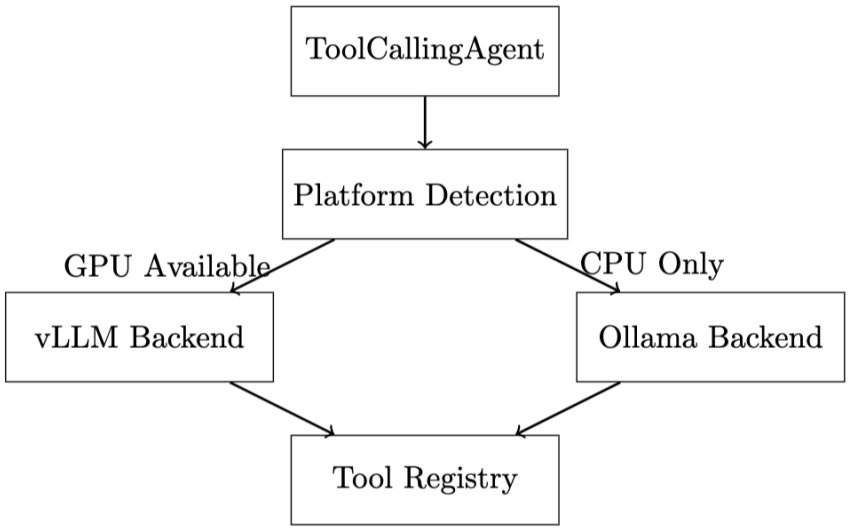

Figure 3. Universal Agent Pattern

B.4 Streaming Architecture

B.4.1 Multi-Type Streaming System

The streaming architecture provides real-time feedback with different content types:

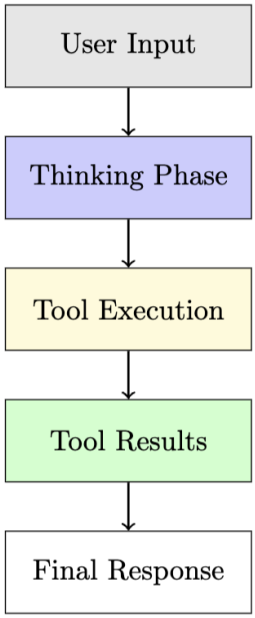

Figure 4. 1 React flow

B.4.2 Streaming Implementation

```
CHUNK_TYPES = {
    "thinking": "Internal reasoning process",
    "tool_call": "Tool execution initiation",
    "tool_result": "Tool execution completion",
    "content": "Final response content",
    "error": "Error notifications"
}

def chat_stream(self, message: str, use_tools: bool = True):
    """Streaming chat with visual feedback."""
    for chunk in self._process_stream(message):
        chunk_type = chunk.get("type")
        content = chunk.get("content", "")

        if chunk_type == "thinking":
            # Gray text for internal reasoning
            print(f"\\033[90m{content}\\033[0m", end="", flush=True)
            yield {"type": "thinking", "content": content}

        elif chunk_type == "tool_call":
            # Blue notification for tool execution
            print(f"Tool: {content['name']}")
            yield {"type": "tool_call", "content": content}

        elif chunk_type == "tool_result":
            # Green result display
            print(f"Result: {content}")
            yield {"type": "tool_result", "content": content}

        elif chunk_type == "content":
```

```
            # Standard text for final response
            print(content, end="", flush=True)
            yield {"type": "content", "content": content}
```

B.4.3 Registry Pattern: Tool Management

Extensible tool system with dynamic registration:

```
class ToolRegistry:
    def __init__(self):
        self.tools = {}
        self._register_builtin_tools()

    def register_tool(self, name: str, function: callable,
                      description: str, parameters: Dict):
        """Register a new tool with OpenAI-compatible schema."""
        self.tools[name] = {
            "function": function,
            "description": description,
            "parameters": parameters
        }

    def execute_tool(self, name: str, arguments: Dict) -> str:
        """Execute a registered tool with error handling."""
        if name not in self.tools:
            return json.dumps({"error": f"Tool {name} not found"})

        try:
            tool_func = self.tools[name]["function"]
            result = tool_func(**arguments)
            return json.dumps({"success": True, "result": result})
        except Exception as e:
            return json.dumps({
                "success": False,
                "error": str(e),
                "traceback": traceback.format_exc()
            })
```

Appendix C Manage the memory of your Claw using Git

To manage the memory, use Git. See Missing Semester Lecture 6.

- **Question 4.** What is the different between `Git`, `GitHub`, `Gitee`, and `Gitlab`.